- **Autograder results for all 6 questions:**

```
Provisional grades
==================
Question q1: 5/5
Question q2: 5/5
Question q3: 10/10
Question q4: 15/15
Question q5: 5/5
Question q6: 9/9
------------------
Total: 49/49
```

- **1. Depth First Search:** Use Stack to create the frontier, the LIFO principle allows DFS to explore one path as deeply as possible.

```
Question q1
===========
*** PASS: test_cases\q1\pacman_1.test
***     pacman layout:          mediumMaze
***     solution length: 130
***     nodes expanded:         146

### Question q1: 5/5 ###


Finished at 2:38:36

Provisional grades
==================
Question q1: 5/5
------------------
Total: 5/5
```

- **2. Breadth First Search:** Use Queue to create the frontier, the FIFO principle allows BFS to explore one path as wide as possible.

```
Question q2
===========
*** PASS: test_cases\q2\pacman_1.test
***     pacman layout:          mediumMaze
***     solution length: 68
***     nodes expanded:         269

### Question q2: 5/5 ###


Finished at 2:39:06

Provisional grades
==================
Question q2: 5/5
------------------
Total: 5/5
```

- **3. Uniform Cost Search:** Use Priority Queue to create the frontier, it allows us to search the node with the least cost because it sets priority for each node.

```
Question q3
===========
*** PASS: test_cases\q3\ucs_4_testSearch.test
***     pacman layout:          testSearch
***     solution length: 7
***     nodes expanded:         14
*** PASS: test_cases\q3\ucs_5_goalAtDequeue.test
***     solution:               ['1:A->B', '0:B->C', '0:C->G']
***     expanded_states:        ['A', 'B', 'C']

### Question q3: 10/10 ###


Finished at 2:39:24

Provisional grades
==================
Question q3: 10/10
------------------
Total: 10/10
```

- **4. A* Search (null Heuristic):** Use Priority Queue With Function to create its frontier, the priority function will calculate the lowest path cost combining the heuristic to prioritize the node.

```
Question q4
===========
*** PASS: test_cases\q4\astar_0.test
***     solution:               ['Right', 'Down', 'Down']
***     expanded_states:        ['A', 'B', 'D', 'C', 'G']

### Question q4: 15/15 ###


Finished at 2:40:03

Provisional grades
==================
Question q4: 15/15
------------------
Total: 15/15
```

- **5. Breadth First Search (Finding all the Corners):** Each state is a tuple of the state's position and the 4 corners' coordinates. The start state is the starting and 4 corners' position. The goal is to check if there are no more corners left. Successors will check if the next position hits the wall and append corners.

```
Question q2
===========
*** PASS: test_cases\q2\pacman_1.test
***     pacman layout:          mediumMaze
***     solution length: 68
***     nodes expanded:         269

### Question q2: 5/5 ###


Question q5
===========
*** PASS: test_cases\q5\corner_tiny_corner.test
***     pacman layout:          tinyCorner
***     solution length:                28

### Question q5: 5/5 ###


Finished at 2:40:21

Provisional grades
==================
Question q2: 5/5
Question q5: 5/5
------------------
Total: 10/10
```

- **6. A* Search (Corners Problem: Heuristic):** Loop through all corners and calculate minimum Manhattan Distance between the current position and remaining corners, pop out the nearest corner, update current position to that nearest corner, and continue to calculate the nearest Manhattan Distance till all corners have been visited.

```
Question q4
===========
*** PASS: test_cases\q4\astar_0.test
***     solution:               ['Right', 'Down', 'Down']
***     expanded_states:        ['A', 'B', 'D', 'C', 'G']
===========
===========
*** PASS: heuristic value less than true cost at start state
path: ['North', 'East', 'East', 'East', 'East', 'North', 'North', 'West', 'West', 'West', 'West', 'North', 'North', 'Nor
th', 'North', 'North', 'North', 'North', 'North', 'West', 'West', 'West', 'West', 'South', 'South', 'East', 'East', 'Eas
t', 'East', 'South', 'South', 'South', 'South', 'South', 'South', 'West', 'West', 'South', 'South', 'South', 'West', 'We
st', 'East', 'East', 'North', 'North', 'North', 'East', 'East', 'East', 'East', 'East', 'East', 'East', 'East', 'South',
 'South', 'East', 'East', 'East', 'East', 'East', 'North', 'North', 'East', 'East', 'North', 'North', 'East', 'East', 'N
orth', 'North', 'East', 'East', 'East', 'East', 'South', 'South', 'South', 'South', 'East', 'East', 'North', 'North', 'E
ast', 'East', 'South', 'South', 'South', 'South', 'South', 'North', 'North', 'North', 'North', 'North', 'North', 'North'
, 'West', 'West', 'North', 'North', 'East', 'East', 'North', 'North']
path length: 106
*** PASS: Heuristic resulted in expansion of 692 nodes

### Question q6: 9/9 ###


Finished at 2:41:54

Provisional grades
==================
Question q4: 15/15
Question q6: 9/9
------------------
Total: 24/24
```

- **Difference between Uniform Cost Search and A* Contours:**
  UCS expands nodes in all directions, which will look like circles getting larger and larger till it reach the goal. In contrast, A* Contours expands nodes more efficiently towards the goal due to the heuristic. Therefore, in larger graphs, A* will be more efficient than Uniform Cost Search.

- **Admissibility Heuristic:**
  An admissible heuristic is a heuristic function that estimates the cost of the cheapest path from any node to the goal (and non-negative). The defining characteristic of an admissible heuristic is that it never overestimates the true cost to reach the goal. It is optimistic because it never suggests a solution that is worse than the actual optimal solution.